JVM

垃圾回收

JVM检测堆内存不足 新的对象无法分配空间 自动触发垃圾回收

可以手动

判断垃圾的方法

引用计数法和可达性分析算法

引用计数法

比较简单 计数器 有引用就加一 引用失效减1， 为0可以被回收

缺点：无法解决循环引用的问题（联想Spring的循环引用）

可达性分析算法

从一组“根对象”开始，沿着引用关系遍历整个对象图，标记所有可达的对象。

不可达的对象 会被认为是“垃圾”，可供回收。

key:要注意根对象的细节

> 根对象集合

1、虚拟机栈（栈帧中的本地变量表）中正在引用的对象

方法执行时，保存在栈上的局部变量（即方法内的变量）如果引用了对象，该对象是可达的。

2、本地方法栈中正在引用的对象：例如 `JNI` 代码中的 `native` 方法持有的对象。

3、静态属性引用的对象：例如类的 `static` 变量如果持有对象引用，该对象是可达的。

4、方法区常量引用的对象:比如 `String` 的常量池中的引用对象。

垃圾回收算法

标记清除算法

标记-清除（Mark-and-Sweep）算法分为“（可达性分析算法）标记（Mark）”和“清除（Sweep）”阶段：首先标记出所有不需要回收的对象，在标记完成后统一回收掉所有没有被标记的对象。

问题：

1. 效率问题：标记和清除两个过程效率都不高。
2. 空间问题：标记清除后会产生大量不连续的内存碎片。

复制算法

为了解决标记-清除算法的效率和内存碎片问题，复制（Copying）收集算法出现了。它可以将内存分为大小相同的两块，每次使用其中的一块。当这一块的内存使用完后，就将还存活的对象复制到另一块去，然后再把使用的空间一次清理掉。这样就使每次的内存回收都是对内存区间的一半进行回收

缺点：

- 可用内存变小：可用内存缩小为原来的一半。
- 不适合老年代：如果存活对象数量比较大，复制性能会变得很差。

> 为什么不适合老年代

老年代对象存活率高（大部分对象长期存活），如果使用复制算法，每次 GC 需要复制大量对象，导致 复制成本太高，影响性能。

但老年代的空间本来就比较紧张，不能浪费一半来做复制。

标记整理算法

标记-整理（Mark-and-Compact）算法是根据老年代的特点提出的一种标记算法，标记过程仍然与“标记-清除”算法一样，但后续步骤不是直接对可回收对象回收，而是让所有存活的对象向一端移动，然后直接清理掉端边界以外的内存。

由于多了整理这一步，因此效率也不高，适合老年代这种垃圾回收频率不是很高的场景。

分代收集算法

在新生代中，每次收集都会有大量对象死去，所以可以选择“复制”算法

老年代的对象存活几率是比较高的，而且没有额外的空间对它进行分配担保，所以我们必须选择“标记-清除”或“标记-整理”算法进行垃圾收集。

垃圾回收器

主要是两个 CMS和G1

还有，

Parallel GC（JDK8默认）：适合高吞吐，批处理场景；STW 较长， 每次 GC 都是 STW。标记-复制（新生代） + 标记-压缩（老年代）

Serial GC (串行垃圾收集器)：单线程执行 GC，适用于客户端模式或单核 CPU 环境

CMS

在JAVA14被移除了

以获取最短回收停顿时间为目标的收集器

第一次实现了让垃圾收集线程与用户线程（基本上）同时工作。

初始标记： 短暂停顿，标记直接与 root 相连的对象（根对象）；

并发标记： 同时开启 **GC** 和用户线程，用一个闭包结构去记录可达对象。但在这个阶段结束，这个闭包结构并不能保证包含当前所有的可达对象。因为用户线程可能会不断的更新引用域，所以 GC 线程无法保证可达性分析的实时性。所以这个算法里会跟踪记录这些发生引用更新的地方。

重新标记： 重新标记阶段就是为了修正并发标记期间因为用户程序继续运行而导致标记产生变动的那一部分对象的标记记录，这个阶段的停顿时间一般会比初始标记阶段的时间稍长，远远比并发标记阶段时间短

并发清除： 开启用户线程，同时 GC 线程开始对未标记的区域做清扫。

用户线程一开一闭一开一闭， 两个并发过程会抢CPU

会产生“内存碎片”（使用标记-清除算法）

CMS在老年代，要配合其他回收器（新生代）

G1

G1是JDK9以后默认的回收器

范围是老年代和新生代，用它一个就行

G1 收集器的运作大致分为以下几个步骤：

- 初始标记： 短暂停顿（Stop-The-World，STW），标记从 GC Roots 可直接引用的对象，即标记所有直接可达的活跃对象
- 并发标记：与应用并发运行，标记所有可达对象。 这一阶段可能持续较长时间，取决于堆的大小和对象的数量。
- 最终标记： 短暂停顿（STW），处理并发标记阶段结束后残留的少量未处理的引用变更。
- 筛选回收：根据标记结果，选择回收价值高的区域，复制存活对象到新区域，回收旧区域内存。这一阶段包含一个或多个停顿（STW），具体取决于回收的复杂度。（不会有内存碎片）

在筛选回收阶段，**G1** 收集器在后台维护了一个优先列表，每次根据允许的收集时间，优先选择回收价值最大的 **Region(**这也就是它的名字 **Garbage-First** 的由来**)** ，在 Mixed GC 或 Young GC 时，会选出一批 Region（回收价值高的），然后把其中的存活对象复制（**Evacuate**）到其他空 **Region**。回收完的 Region 直接整块清空，变成 Free Region。

区别？

CMS是老年代的,要配合其他收集器；G1是老年代和新生代，用它一个就行

CMS是以最小的停顿时间为目标的（并行标记和并行清除）；而G1是可预测垃圾回收的停顿时间，回收时只选部分“垃圾多”的块集中处理

CMS是标记清除，有碎片；G1是标记整理，没碎片。

CMS第四阶段是并发清除（标记清除）；G1第四阶段是筛选回收，分region清除，标记整理。

CMS有浮动垃圾问题，G1解决了。看下面

CMS浮动垃圾问题？

浮动垃圾 =“本轮 **GC** 无法回收，但实际上已经死亡的对象”。

说白了就是并发问题

是指在并发标记和并发清理阶段，由于用户线程继续运行而新产生的垃圾对象，这些对象无法在当前GC周期被回收，只能留到下一次**GC**处理。

如对象引用被断开，**CMS**标记**A-》B**时，只标了**A**但是**AB** 断开了，**B**就失联了，**B**是垃圾 但是被标为可达 产生浮动垃圾

CMS的设计目标是减少**STW**（**Stop-The-World**）停顿时间，所以它允许用户线程在标记和清理阶段并发执行。但这也带来了副作用

重新标记（Remark）只能修正在并发标记期间新建或新增引用的对象（防止漏标活对象），但它不会去“反标记”那些已经标记了、但引用断开的对象。

> 为什么是问题？下一轮解决就好了啊？

原因是类似于求极限，每次清楚的量不达预期，慢慢的就逼近内存极限了，类似C盘空间。

> *G1*怎么解决的

SATB（Snapshot-At-The-Beginning）算法

1. 在并发标记开始时，对整个堆做一个快照（Snapshot），之后所有新分配的对象默认视为存活。
2. 在最终标记（**Remark**）阶段，只处理引用变化的部分，确保所有垃圾被正确识别。
3. 在筛选回收（**Evacuation**）阶段，直接清理未被标记的对象，不会遗漏浮动垃圾。

比CMS少很多。只是缓解罢了，引用变化的部分能被清理

如果应用线程断开了 A→B 的引用，G1 会通过 写屏障（**Write Barrier**） 把“旧引用 B”记录下来，
👉 这保证了标记结果相当于“快照时刻”的状态。
👉 所以即使 A→B 断开了，G1 仍会标记 B，因为在快照时它是可达的。

G1 通过 SATB 快照机制，让标记结果更一致、更接近真实状态，从而大幅减少“错标为活”的对象，提升空间利用率和延迟稳定性。

因为 CMS 的并发标记不稳定，随着应用写入越频繁，浮动垃圾越多。

G1 的 SATB 快照机制保证标记结果一致，

轻重**GC**与内存区

堆内存细分三个区域：新生区（伊甸园区）、老年区、永久区

新生区：伊甸园、幸存区0区、幸存区1区　动态概念　轻GC后没有被回收 进入幸存区　0和1区会交互

> *0*区和*1*区怎么交互的

Survivor 0区（S0）和 Survivor 1区（S1）*是新生代（*Young Generation*）的重要组成部分，主要用于存放*在Minor GC后仍然存活的对象。它们的交互方式遵循复制算法（Copying Algorithm）**

0和1左右弹跳 复制算法

对象在新生代中 Eden 区分配。当 Eden 区没有足够空间进行分配时，虚拟机将发起一次 Minor GC。

假设GC次数为20 （默认15次）

超过阈值还是活下来的去养老区（幸存者区满了也会去）

Major GC 主要针对老年代进行回收，但不一定只回收老年代。　当老年代空间不足时，或者系统检测到年轻代对象晋升到老年代的速度过快，可能会触发Major GC。

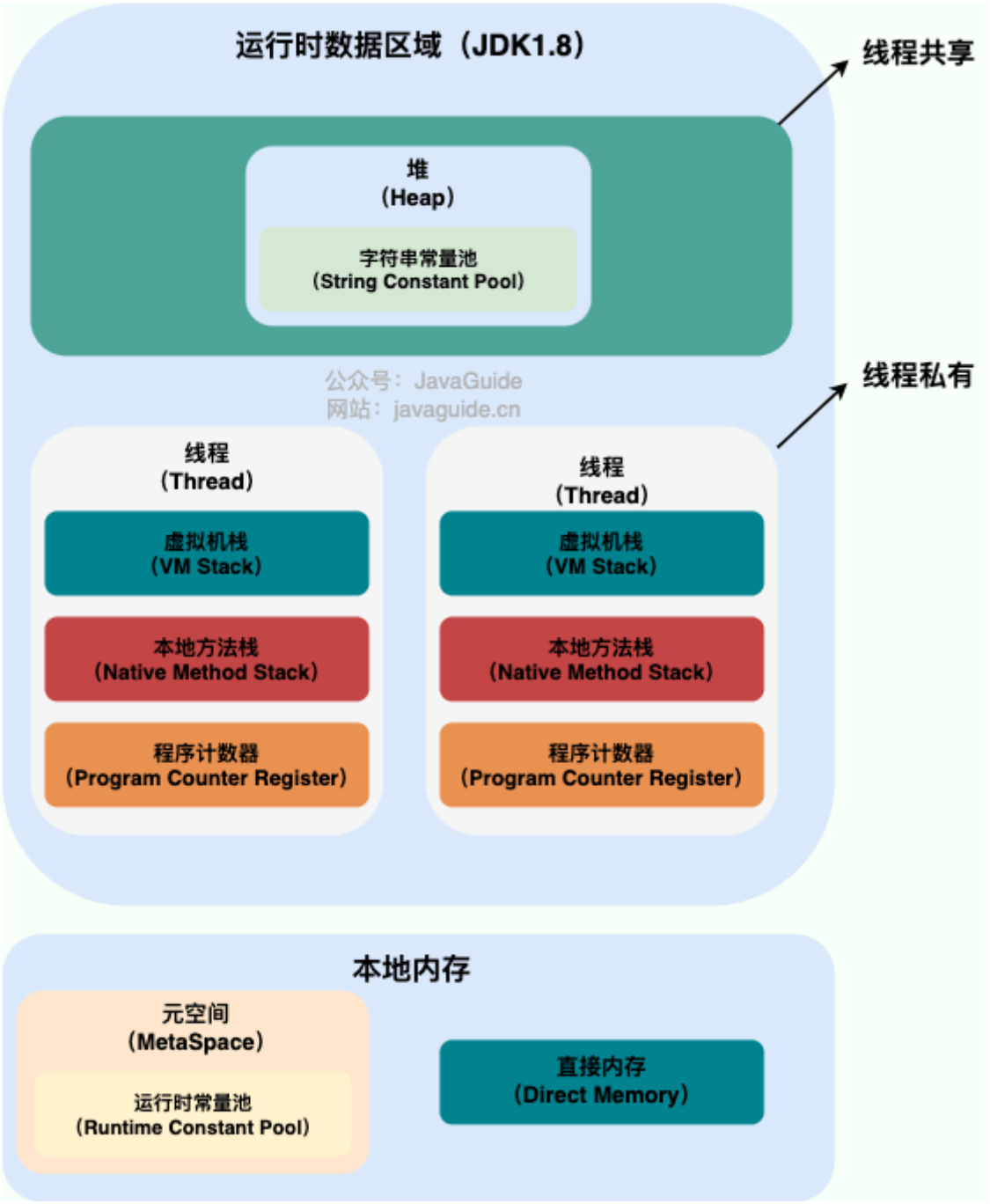
大对象会分配到老年代或者大对象区，防止新生代空间不足/内存碎片

养老区满了会触发重量级GC (full GC)，对整个堆内存（包括年轻代、老年代以及永久代/元空间）进行回收。

永久区（JDK8以后叫元空间）

这个区域常驻内存的。用来存放JDK自身携带的Class对象。Interface元数据，存储的是运行时的一些环境 这个区域不存在垃圾回收！ 关闭虚拟机时就会释放

内存模型



> 程序计数器

当前线程正在执行的JVM指令地址，如果正在执行的native方法，计数器值为null。 线程私有 ，生命周期与线程相同

> *Java*虚拟机栈

每个方法在执行时都会创建一个栈帧，每个栈帧存储局部变量表，操作数栈、方法出口。理解成被调用方法即可。 线程私有，生命周期与线程相同

> 本地方法栈

与虚拟机栈类似，主要为虚拟机使用到的native方法服务，在HOTSPOT里面 与Java虚拟机栈合二为一。 线程私有，生命周期与线程相同

> *Java*堆

线程共享，在虚拟机启动时创建

分为新生代和老年代。上面已详细说明

主要存放新建的对象实例和数组

> 方法区（元空间）

存储已经被虚拟机加载的类信息、常量、静态变量等数据

方法的 字节码

方法的 符号引用（指向字段、类、其他方法）

方法的 修饰符、返回类型、参数表

永久代被元空间取代了（JDK8）

线程共享

> 运行时常量池

方法区（元空间）的一部分，用于存放编译期生成的各种字面量和符号引用

常见考题：字符串还有各种类的常量池

String s = new String（“abc”）

先在常量池找abc 如果无，会创建两次，分别放堆和常量池

String s1 = "hello"; //不new的话会放到常量池 ， new的话会放常量池和堆

调用 `intern()`，会返回 常量池中的对象引用

Integer 缓存池（**Integer Cache**）

JVM 默认缓存 **-128** 到 **127** 的 **Integer** 对象

JVM 启动时创建Integer缓存对象，在堆上

Integer i1 = new Integer(100); //会在堆上创建一个新的 Integer 对象

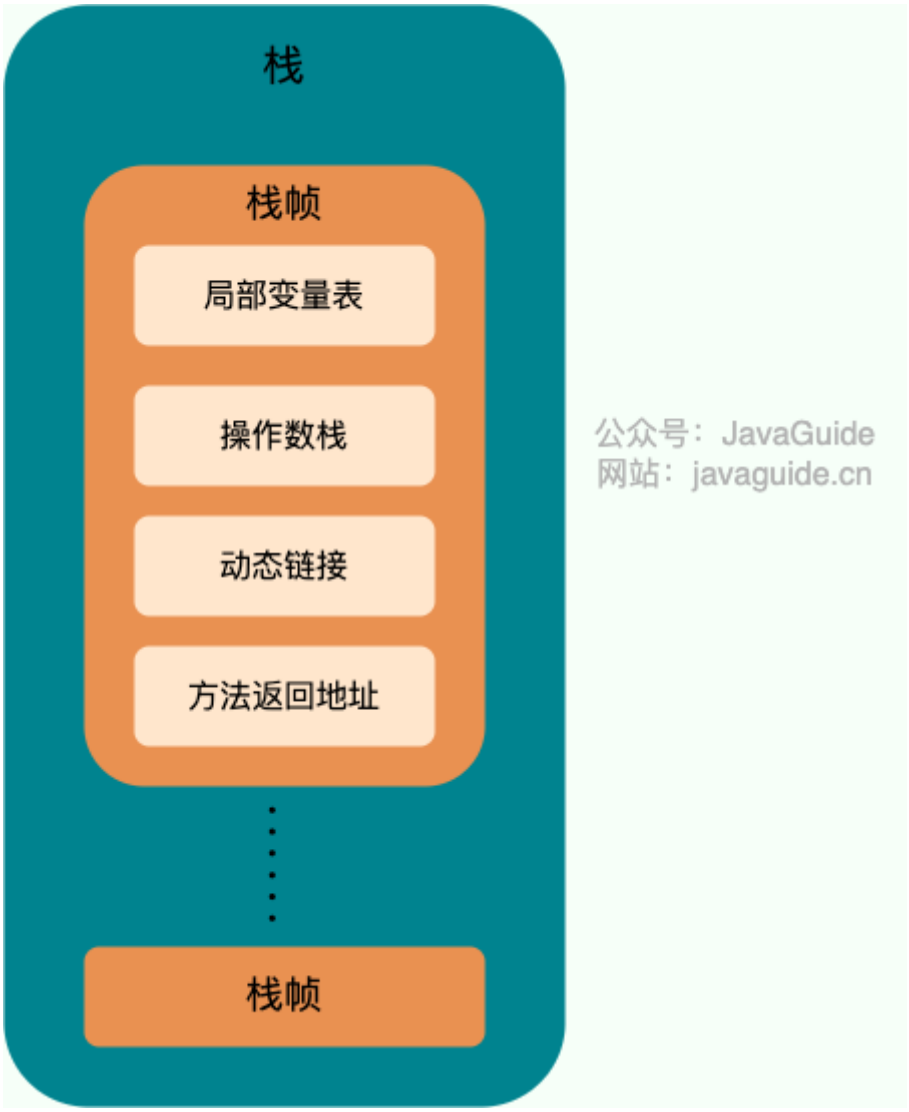
> 直接内存

堆外内存，显著提高I/O性能

栈

Java 虚拟机栈（后文简称栈）也是线程私有的，它的生命周期和线程相同，随着线程的创建而创建，随着线程的死亡而死亡。

每个栈帧中都拥有：局部变量表、操作数栈、动态链接、方法返回地址。和数据结构上的栈类似，两者都是先进后出的数据结构，只支持出栈和入栈两种操作。



一些八股题

> 方法区中的方法的执行过程？

方法调用

- JVM 在 栈帧（**Stack Frame**） 中创建一个新的帧
- 栈帧包含：
    - 局部变量表（参数、局部变量）
    - 操作数栈（执行运算的临时空间）
    - 动态链接（指向方法区中方法的引用）
    - 方法返回地址

解析符号引用

- 栈帧中调用方法时，可能需要访问方法区中的符号引用

- JVM 将符号引用解析成实际内存地址（运行时常量池 + 方法区）

执行字节码

- JVM 解释器执行方法区的字节码，或者 JIT 编译成机器码执行
- 操作数栈进行运算，局部变量表存储中间结果

方法返回

- 返回值（如果有）放入调用者的操作数栈
- 栈帧出栈，控制权返回调用者

> 什么是符号引用

编译期生成的对类、方法或字段的引用，它 不是直接的内存地址，而是一个“名字”或“符号”，JVM 在运行时需要将其解析成 直接引用（**Direct Reference**）。

> 强软弱虚引用

倒背如流

> 内存溢出的情况？

堆内存溢出

栈溢出

元空间溢出

直接内存内存溢出

> 内存泄漏

第一种情况，对象不再使用，但仍然有引用，GC 回收不了

静态属性过多，只要类还在（通常整个应用周期），GC 就 回收不了这些对象

static 变量属于类，放在元空间了

第二种情况，未关闭的资源，使用try-with-resources关闭

第三种ThreadLocal

类加载过程

> 为什么要类加载

Java 程序写完后，`.java` 文件会编译成 `.class` 文件（字节码），
👉 但 **JVM** 不能直接用这个文件，必须先把 `.class` 里的内容“翻译”成 JVM 能识别、能运行的类模板（**Class**对象），

把类的字节码文件（**.class**）加载到内存中，并在方法区中生成对应的 **Class** 对象，供 **JVM** 在运行时使用。

> 创建对象的过程

一、检查类是否已经被加载。

二、分配内存空间

三、初始化零值

四、必要设置，对象头

对象的哈希码，对象的GC分代年龄，元数据信息。

五，执行init方法

> 类加载过程

检查方法区是否已加载

- 如果类还没加载，JVM 找到 `.class` 文件
- 读取字节码，生成 **Class** 对象，放到 方法区/元空间

链接（**Linking**）

- 验证：检查字节码是否正确
- 准备：给静态变量分配内存并赋默认值
- 解析：符号引用解析成直接引用

初始化（**Initialization**）

- 执行静态变量的显式赋值和静态代码块
- 完成类的初始化

> 类加载器

从上到下

启动类加载器 **Bootstrap Class Loader**：最顶层的类加载器，负责加载Java的核心库，位于rt.jar中的类，C++实现

扩展类加速器 **Extension Class Loader**：继承自ClassLoader类，负载加载Java扩展目录，父加载器就是启动类加载器,Java语言实现

系统类加载器/应用程序类加载器 Appclassloader ：Java语言实现,负责加载用户类路径上的指定类库，平常编写时默认使用的类加载器

自定义类加载器：扩展灵活性和安全性。

> 类加载器的作用？

类加载器（ClassLoader）是 JVM 中用于实现类的动态加载的组件。

java文件编译后会生成class

字节码是 `.class` 文件里保存的 **JVM** 指令，是 **Java** 源代码的中间形式。

Java 的类加载器都继承自 `java.lang.ClassLoader`。
核心就是重写其中的 `findClass()` 方法👇

`findClass()` 的职责其实就是：
👉 找到目标类的「字节码」并返回 **Class** 对象（通过 **defineClass** 实现）。

`defineClass()` 是 JVM 的 native 方法，它会：

- 校验字节码是否合法
- 解析常量池、方法、字段等信息
- 生成 JVM 方法区里的 **Class** 对象

类加载器必须先“读字节码”到内存，再交给 JVM，才能让 JVM 知道类的内容并执行它。

> 双亲委派

保证类的唯一性，确保所有加载请求会传递到启动类加载器（最顶层），避免了不同类加载器重复加载相同类的情况，保证了唯一性。

保证安全性：只加载信任的类路径中的类

支持隔离和层次划分：实现沙箱安全机制，保证各个层级类加载器的职责清晰。

简化了加载流程：大部分类能够被正确的类加载器加载。

当一个类加载器收到类加载请求时，它不会自己立刻去加载，而是先把这个请求交给它的父加载器，一层层往上交，直到最顶层（启动类加载器）去尝试加载。
👉 如果父加载器加载不了，才由子加载器自己去加载。

和DNS完全反过来

```java
public Class<?> loadClass(String name) throws
ClassNotFoundException {
    // 1 检查自己是否已经加载过
    Class<?> c = findLoadedClass(name);
    if (c != null) {
        return c;
    }

    // 2 委派给父加载器
    ClassLoader parent = this.getParent();
    if (parent != null) {
        try {
            c = parent.loadClass(name); // <--- 核心：委派
            return c;
        } catch (ClassNotFoundException e) {
            // 父加载器找不到，自己加载
        }
    }

    // 3 自己加载
    return findClass(name);
}
```

`loadClass()` 负责委派和入口控制 → `findClass()` 负责找字节码 → `defineClass()` 负责把字节码交给 JVM 生成类结构

> 破坏双亲委派的例子？

Tomcat需要同时加载不同Web应用中“相同类名”的不同版本，还要支持应用之间隔离。

让每个 WebApp 可以加载自己目录下的 `WEB-INF/classes` 和 `WEB-INF/lib` 中的类

👉 **Tomcat** 最核心的“破坏双亲委派”点在于 `WebAppClassLoader`
👉 它的 `loadClass()` 方法被重写，优先自己加载应用内的类，找不到再委托父加载器，这正好和双亲委派相反！

JVM指令和调优

Jstack 和jstat收集数据，如内存不足，频繁GC，停顿时间过长。

停顿时间过长用G1/CMS，不用parallel

Full GC 是否频繁（比如老年代顶满），内存使用是否合理（新生代 vs 老年代）

这种情况的话条新生代和老年代的比例大小 -XX:NewRatio 或者调整个堆的大小

```
-Xms /  -Xmx      初始 / 最大堆大小（建议设成一样，避免动态扩容）
```

调 G1 region大小

```
-XX:G1HeapRegionSize
```